

Cybol Tutorial: neue Formate, Typen und Funktionen anlegen

Inhaltsverzeichnis

Cybol Tutorial: neue Formate, Typen und Funktionen anlegen.....	1
1. Ein neues Format.....	1
1.1 Ein Cybol Beispiel.....	1
1.2 Konstanten anlegen.....	2
1.3 Einlesen des neuen Formates.....	2
1.4 Ausgabe.....	3
2. Einen Typ anlegen.....	4
2.1 Konstanten anlegen.....	4
2.2 Einlesen des neuen Types.....	5
2.3 Ausgabe.....	5
3. Eine neue Funktion anlegen.....	6
3.1 Beispiel erweitern.....	6
3.2 Konstanten definieren.....	7
3.3 Funktion einlesen.....	8
3.4 Funktion ausgeben.....	8
4.5 Funktion aufrufen.....	9
4.6 Die Funktion.....	9

1. Ein neues Format

In ersten Teil dieses Tutorials soll gezeigt werden, wie man in Cybol ein neues Format einführt. Das neue Format wird „custom/euro“ genannt. Es sollen Beträge eingelesen werden (zum Beispiel „120,50“) und ausgegeben werden mit Währungskennzeichnung (Zum Beispiel „120,50 EUR“)

1.1 Ein Cybol Beispiel

Zunächst wird ein Beispiel angelegt. In diesem wird ein Knoten namens „betrag“ erzeugt, welcher das Format „custom/euro“ bekommt. Anschließend wird der Knoten mit dem Wert „120,50“ initialisiert und auf der Konsole ausgegeben.

```
<model>
  <part name="create_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="memorise/create" model="">
    <property name="name" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="betrag"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="element" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="part"/>
  </part>

  <part name="initialise_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="communicate/receive" model="">
    <property name="channel" channel="inline" encoding="utf-8" language="text/cybol" format="meta/channel" model="inline"/>
    <property name="encoding" channel="inline" encoding="utf-8" language="text/cybol" format="meta/encoding" model="utf-8"/>
    <property name="language" channel="inline" encoding="utf-8" language="text/cybol" format="meta/language" model="text/cybol"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="message" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="120,50"/>
    <property name="model" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
  </part>

  <part name="print_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="communicate/send" model="">
    <property name="channel" channel="inline" encoding="utf-8" language="text/cybol" format="meta/channel" model="terminal"/>
    <property name="encoding" channel="inline" encoding="utf-8" language="text/cybol" format="meta/encoding" model="utf-8"/>
    <property name="language" channel="inline" encoding="utf-8" language="text/cybol" format="meta/language" model="text/cybol"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="message" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
    <property name="new_line" channel="inline" encoding="utf-8" language="text/cybol" format="logicvalue/boolean" model="true"/>
  </part>

  <part name="exit_application" channel="inline" encoding="utf-8" language="text/cybol" format="live/exit" model="">
  </part>
</model>
```

Datei: examples/euro/run.cybol

Zunächst wird beim Ausführen des Beispiels kein Wert auf der Konsole angezeigt, da wir den Interpreter das neue Format „custom/euro“ noch nicht beigebracht haben.

1.2 Konstanten anlegen

Zu erst wird für den Interpreter eine neue Konstante angelegt. Diese ist ein Zahlenwert, welcher später das neue Format repräsentiert. Dafür wird die Datei „state_cyboi_format.c“ um folgenden Quelltext erweitert.

```
//
// custom
//

/** The euro custom state cyboi format. */
static int* EURO_CUSTOM_STATE_CYBOI_FORMAT = NUMBER_430_INTEGER_STATE_CYBOI_MODEL_ARRAY;
```

Datei: src/constant/format/cyboi/state_cyboi_format.c

Nun muss noch für den Interpreter eine Cybol Konstante angelegt werden. Über diese wird später anhand der Zeichenfolge „custom/euro“ das neue Format erkannt. Hierfür wird eine Neue Datei namens „custom_state_cybol_format.c“ angelegt.

```
#ifndef CUSTOM_STATE_CYBOL_FORMAT_CONSTANT_SOURCE
#define CUSTOM_STATE_CYBOL_FORMAT_CONSTANT_SOURCE

#include <stddef.h>

#include "../constant/model/cyboi/state/integer_state_cyboi_model.c"

static wchar_t EURO_CUSTOM_STATE_CYBOL_FORMAT_ARRAY[] = {L'c', L'u', L's', L't', L'o', L'm', L',', L'e', L'u', L'r', L'o'};
static wchar_t* EURO_CUSTOM_STATE_CYBOL_FORMAT = EURO_CUSTOM_STATE_CYBOL_FORMAT_ARRAY;
static int* EURO_CUSTOM_STATE_CYBOL_FORMAT_COUNT = NUMBER_11_INTEGER_STATE_CYBOI_MODEL_ARRAY;

/* CUSTOM_STATE_CYBOL_FORMAT_CONSTANT_SOURCE */
#endif
```

Datei: src/constant/format/cybol/state/custom_state_cybol_format.c

Der Wert „_COUNT“ repräsentiert die Anzahl der Buchstaben des Formates.

1.3 Einlesen des neuen Formates

Zunächst muss dem Format-Deserialisierer angepasst werden. Die Folgenden Zeilen werden in die Datei „format_cybol_deserialiser.c“ eingefügt. Sie bewirken, dass der eingelesene Wert für das Format mit der Cybol-Konstante verglichen wird, welche den Wert „custom/euro“ enthält. Ist der eingelesene Wert mit diesem identisch, wurde das Format erkannt und es wird in den internen Baum der Wert der Cyboi-Konstante.

```
...
#include "../constant/format/cybol/state/custom_state_cybol_format.c"
...
//
// custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_all_array((void*) &r, p1, (void*) EURO_CUSTOM_STATE_CYBOL_FORMAT,
                      (void*) EQUAL_COMPARE_LOGIC_CYBOI_FORMAT, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
                      p2, (void*) EURO_CUSTOM_STATE_CYBOL_FORMAT_COUNT);
```

```

if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    overwrite_item_element(p0, (void*) EURO_CUSTOM_STATE_CYBOI_FORMAT,
        (void*) INTEGER_NUMBER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
        (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME,
        (void*) TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);
}
}

```

Datei: src/executor/representer/deserialiser/cybol/format_cybol_deserialiser.c

Nach dem nun der Format-Deserialisierer unser neues Format erkennt muss nun dem Cybol-Deserialisierer erklärt werden, wie es mit den Daten, welche dieses neue Format besitzen, umgehen soll (im oberen Beispiel haben wir als Daten „120,50“ angegeben). Hierfür muss die Datei „cybol_deserialiser.c“ angepasst werden. Für unser Beispiel sollen Daten, welche das Format „custom/euro“ besitzen, ohne weitere Bearbeitung (also als einfache Zeichenkette) in den Baum übernommen werden. Man könnte an dieser Stelle auch eine Funktion aufrufen und die Daten noch formatieren oder überprüfen.

```

//
// custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {
    compare_integer_equal((void*) &r, p4, (void*) EURO_CUSTOM_STATE_CYBOI_FORMAT);

    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

        overwrite_item_element(p0, p2, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE, p3,
            (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL,
            (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL,
            (void*) TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);

    }
}
}

```

Datei: src/executor/representer/deserialiser/cybol/cybol_deserialiser.c

1.4 Ausgabe

Nach dem nun Werte, welche das neue Format besitzen, eingelesen werden können, würde bei Ausführung des oben erstellten Beispiels trotzdem noch keine Ausgabe erfolgen, da wir den Interpreter noch nicht erklärt haben, wie er Werte des neuen Formates serialisieren soll.

Zunächst erklären wir den Format-Serialisierer, wie er das neue Format „custom/euro“ ausgeben soll. Dies hat keine Bedeutung für unser Beispiel-Programm, ist allerdings wichtig für andere Funktionen, welche zum Beispiel den Baum in einer Textdatei-Ausgabe können. Auch ohne diese Anpassung würde unser Programm später funktionieren. Jedoch kann es sein, dass Entwickler sich zur Fehlersuche den Baum in eine Text-Datei serialisieren lassen. Ohne diese Anpassung würde dann das neue Format nicht im Baum auftauchen.

```

..
#include "../.../constant/format/cybol/state/custom_state_cybol_format.c"
..
//
// custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_integer_equal((void*) &r, p1, (void*) EURO_CUSTOM_STATE_CYBOI_FORMAT);
}
}

```

```

if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    append_item_element(p0, (void*) EURO_CUSTOM_STATE_CYBOL_FORMAT,
        (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
        (void*) EURO_CUSTOM_STATE_CYBOL_FORMAT_COUNT,
        (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

}
}

```

Datei: Src/executor/representer/serialiser/cybol/format_cybol_serialiser.c

Nun geht es an die eigentliche Serialisierung. Es sollen immer das Währungskennzeichen „EUR“ hinter dem Betrag stehen. Hierfür muss die Datei „cybol_serialiser.c“ angepasst werden.

```

//
// custom
//
if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_integer_equal((void*) &r, p5, (void*) EURO_CUSTOM_STATE_CYBOL_FORMAT);

    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {
        //Wert kann hier individuell bearbeitet werden
        wchar_t EURO_CUSTOM_SYMBOL_ARRAY[] = {L' ',L'E', L'U', L'R'};
        wchar_t* EURO_CUSTOM_SYMBOL = EURO_CUSTOM_SYMBOL_ARRAY;
        append_item_element(p0, p1, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE, p2,
            (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);
        append_item_element(p0, (void*) EURO_CUSTOM_SYMBOL,
            (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
            (void*) NUMBER_4_INTEGER_STATE_CYBOI_MODEL_ARRAY,
            (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

    }

}

```

Datei: Src/executor/representer/serialiser/cybol/cybol_serialiser.c

Nun wird der Wert ausgegeben:

```

120,50 EUR
Information: Exit cyboi normally.

```

2. Einen Typ anlegen

Dieser teil des Tutorials soll zeigen, wie man einen neuen Typ anlegt. Der Typ soll später von einer Funktion verwendet werden. Der neue Type heißt „custom/vat“. Typen werden für interne Zwecke benötigt.

Die Besonderheit bei Typen ist, das man diese zu erst wie ein Format definieren muss.

Im Gegensatz zu einem normalen Format wird jedoch keine Deserialisierung (Datei „cybol_deserialiser.c“) und Serialisierung (Datei „cybol_serialiser.c“) benötigt, da es bei Typen keinen Wert gibt, welcher umgewandelt werden kann. Es müssen also lediglich die Konstanten angelegt werden und die Format-Deserialisierung (Datei „format_cybol_deserialiser.c“).

2.1 Konstanten anlegen

Da Typen eine Format als Grundlage haben muss lediglich eine Konstante für den neuen Typ angelegt werden. Dafür wird die Datei „state_cyboi_type.c“ um folgende Zeilen erweitert.

```

//
// custom
//

```

```
/** The xyz custom state cyboi type. */  
static int* VAT_CUSTOM_STATE_CYBOI_TYPE = NUMBER_62_INTEGER_STATE_CYBOI_MODEL_ARRAY;
```

Datei: src/constant/type/cyboi/state_cyboi_type.c

2.2 Einlesen des neuen Types

Für das einlesen des neuen Types muss lediglich die Datei „type_cybol_deserialiser.c“ angepasst werden. Diese Weist dem Knoten „Typ“ im Baum den Konstanten-Wert der neuen Type-Konstante zu.

```
//  
// custom  
//  
  
if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {  
  
    compare_integer_equal((void*) &r, p1, (void*) VAT_CUSTOM_STATE_CYBOI_FORMAT);  
  
    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {  
  
        overwrite_item_element(p0, (void*) VAT_CUSTOM_STATE_CYBOI_TYPE,  
            (void*) INTEGER_NUMBER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,  
            (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME,  
            (void*) TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);  
    }  
}
```

Datei: src/executor/representer/deserialiser/cybol/type_cybol_deserialiser.c

2.3 Ausgabe

Da Typen ja keinen Wert zugewiesen bekommen haben, welchen man serialisieren könnte benötigt man bei Typen keine Ausgabe-Funktionalitäten.

Allerdings kann man den Namen des Typen selbst serialisieren. Dies ist nur für Entwickler gedacht, welche Beispiels-weiße den Laufzeitbaum zur Fehlersuche in eine Text-Datei ausgeben. Damit hier der neue Typ angezeigt wird, benötigt man einen Eintrag in der Datei „type_cybol_serialiser.c“.

```
//  
// custom  
//  
  
if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {  
  
    compare_integer_equal((void*) &r, p1, (void*) VAT_CUSTOM_STATE_CYBOI_TYPE);  
  
    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {  
  
        overwrite_item_element(p0, (void*) VAT_CUSTOM_STATE_CYBOI_FORMAT,  
            (void*) INTEGER_NUMBER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,  
            (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME,  
            (void*) TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);  
    }  
}
```

Datei: src/executor/representer/serialiser/cybol/type_cybol_serialiser.c

Nun haben wir einen Typ angelegt, welcher bis jetzt keinen nutzen hat, aber im folgenden nun verwendet wird.

3. Eine neue Funktion anlegen

Nun wird eine neuer Funktion angelegt. Es geht hierbei nicht um den Sinn der Funktion, sondern lediglich um eine Demonstration, wie man eine Funktion anlegt.

3.1 Beispiel erweitern

Das im oberen erstellte Beispiel soll nun um eine Funktion erweitert werden. Die Funktion heißt „custom/pretext“ Die Funktion bekommt die Parameter „result“, „pretext“ und „type“.

Result ist der Knoten, in welchen bereits der betrag steht („120,50“)

pretext ist ein Text, welcher vor den Betrag gesetzt werden soll. (zum Beispiel „inklusive“)

type: als Typ kann ein Type angegeben werden, wie „text/plain“ oder „custom/vat“. Ist der Type „custom/vat“ wird Vor den Betrag noch das Wort „Mehrwertsteuer:“ eingefügt.

Ziel des Beispiels ist es, dass bei pretext „inkl.“ und type „custom/vat“ folgender text im result steht: „inkl. Mehrwertsteuer: 120,50“. Wenn der Typ ein anderer ist soll nur „inkl. 120,50“ im result stehen.

Das Beispiel hat lediglich den Sinn Funktionen und deren Parameter zu erläutern und verfolgt keinen besonderen Sinn.

```
<model>
  <part name="create_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="memorise/create" model="">
    <property name="name" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="betrag"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="element" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="part"/>
  </part>

  <part name="initialise_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="communicate/receive" model="">
    <property name="channel" channel="inline" encoding="utf-8" language="text/cybol" format="meta/channel" model="inline"/>
    <property name="encoding" channel="inline" encoding="utf-8" language="text/cybol" format="meta/encoding" model="utf-8"/>
    <property name="language" channel="inline" encoding="utf-8" language="text/cybol" format="meta/language" model="text/cybol"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="message" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="120,50"/>
    <property name="model" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
  </part>

  <part name="do_Pretext" channel="inline" encoding="utf-8" language="text/cybol" format="custom/pretext" model="">
    <property name="result" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
    <property name="pretext" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="inkl"/>
    <property name="type" channel="inline" encoding="utf-8" language="text/cybol" format="meta/type" model="custom/vat"/>
  </part>

  <part name="print_betrag" channel="inline" encoding="utf-8" language="text/cybol" format="communicate/send" model="">
    <property name="channel" channel="inline" encoding="utf-8" language="text/cybol" format="meta/channel" model="terminal"/>
    <property name="encoding" channel="inline" encoding="utf-8" language="text/cybol" format="meta/encoding" model="utf-8"/>
    <property name="language" channel="inline" encoding="utf-8" language="text/cybol" format="meta/language" model="text/cybol"/>
    <property name="format" channel="inline" encoding="utf-8" language="text/cybol" format="meta/format" model="custom/euro"/>
    <property name="message" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
    <property name="new_line" channel="inline" encoding="utf-8" language="text/cybol" format="logicvalue/boolean" model="true"/>
  </part>

  <part name="exit_application" channel="inline" encoding="utf-8" language="text/cybol" format="live/exit" model="">
</model>
```

Datei: examples/euro/run.cybol

3.2 Konstanten definieren

Zunächst wird wieder eine Konstante benötigt, welche eine Zahl bekommt, die die Funktion Repräsentiert.

```
/** CUSTOM */  
static int* PRETEXT_CUSTOM_LOGIC_CYBOI_FORMAT = NUMBER_930_INTEGER_STATE_CYBOI_MODEL_ARRAY;
```

Datei: src/constant/format/cyboi/logic_cyboi_format.c

Anschließend wird noch eine Konstante benötigt, welche die Zeichenfolge „custom/pretext“ enthält und eine Konstante die die Länge der Zeichenfolge enthält (14). Hierfür wird die Datei „custom_logic_cybol_format.c“ angelegt.

```
#ifndef CUSTOM_LOGIC_CYBOL_FORMAT_CONSTANT_SOURCE  
#define CUSTOM_LOGIC_CYBOL_FORMAT_CONSTANT_SOURCE  
  
#include <stddef.h>  
  
#include ".././.././constant/model/cyboi/state/integer_state_cyboi_model.c"  
  
static wchar_t PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT_ARRAY[] =  
    {L'c', L'u', L's', L't', L'o', L'm', L '/', L'p', L'r', L'e', L't', L'e', L'x', L't'};  
  
static wchar_t* PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT = PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT_ARRAY;  
static int* PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT_COUNT = NUMBER_14_INTEGER_STATE_CYBOI_MODEL_ARRAY;  
  
/* CUSTOM_LOGIC_CYBOL_FORMAT_CONSTANT_SOURCE */  
#endif
```

Datei: src/constant/format/cybol/logic/custom_logic_cybol_format.c

Zusätzlich werden noch Konstanten benötigt, welche die Zeichenfolge und deren Längen beinhalten, der Parameter der Funktion. Es werden Konstanten für die Parameter „result“, „pretext“ und „type“ benötigt. Hierfür wird die Datei „custom_operation_cybol_name.c“ angelegt.

```
#ifndef CUSTOM_OPERATION_CYBOL_NAME_CONSTANT_SOURCE  
#define CUSTOM_OPERATION_CYBOL_NAME_CONSTANT_SOURCE  
  
#include <stddef.h>  
  
#include ".././.././constant/model/cyboi/state/integer_state_cyboi_model.c"  
  
/** The result custom operation cybol name. */  
static wchar_t RESULT_CUSTOM_OPERATION_CYBOL_NAME_ARRAY[] = {L'r', L'e', L's', L'u', L't', L't'};  
static wchar_t* RESULT_CUSTOM_OPERATION_CYBOL_NAME = RESULT_CUSTOM_OPERATION_CYBOL_NAME_ARRAY;  
static int* RESULT_CUSTOM_OPERATION_CYBOL_NAME_COUNT = NUMBER_6_INTEGER_STATE_CYBOI_MODEL_ARRAY;  
  
/** The pretext custom operation cybol name. */  
static wchar_t PRETEXT_CUSTOM_OPERATION_CYBOL_NAME_ARRAY[] = {L'p', L'r', L'e', L't', L'e', L'x', L't'};  
static wchar_t* PRETEXT_CUSTOM_OPERATION_CYBOL_NAME = PRETEXT_CUSTOM_OPERATION_CYBOL_NAME_ARRAY;  
static int* PRETEXT_CUSTOM_OPERATION_CYBOL_NAME_COUNT = NUMBER_7_INTEGER_STATE_CYBOI_MODEL_ARRAY;  
  
/** The type custom operation cybol name. */  
static wchar_t TYPE_CUSTOM_OPERATION_CYBOL_NAME_ARRAY[] = {L't', L'y', L'p', L'e'};  
static wchar_t* TYPE_CUSTOM_OPERATION_CYBOL_NAME = TYPE_CUSTOM_OPERATION_CYBOL_NAME_ARRAY;  
static int* TYPE_CUSTOM_OPERATION_CYBOL_NAME_COUNT = NUMBER_4_INTEGER_STATE_CYBOI_MODEL_ARRAY;  
  
/* CUSTOM_OPERATION_CYBOL_NAME_CONSTANT_SOURCE */  
#endif
```

Datei: src/constant/name/cybol/operation/custom_operation_cybol_name.c

3.3 Funktion einlesen

Der Logik-Deserialisierer muss erweitert werden, damit dieser die Zeichenfolge „custom/pretext“ des cybol-Programm-Textes mit der Konstante vergleicht und im Falle einer Übereinstimmung die Cyboi-Konstante der Funktion in den Laufzeitbaum einträgt um später die eigentliche Funktion aufrufen zu können. Dafür wird die Datei „format_cybol_deserialiser.c“ erweitert.

```
...
#include ".././.././././constant/format/cybol/logic/custom_logic_cybol_format.c"
...

//
// Custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_all_array((void*) &r, p1, (void*) PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT,
        (void*) EQUAL_COMPARE_LOGIC_CYBOI_FORMAT, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
        p2, (void*) PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT_COUNT);

    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

        overwrite_item_element(p0, (void*) PRETEXT_CUSTOM_LOGIC_CYBOI_FORMAT,
            (void*) INTEGER_NUMBER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
            (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME,
            (void*) TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);
    }
}
```

Datei: src/executor/representer/deserialiser/cybol/format_cybol_deserialiser.c

3.4 Funktion ausgeben

Da eine Funktion keine Serialisierung besitzt, wird auch hier lediglich zum Zwecke der Fehlersuche die Datei „format_cybol_serialiser.c“ erweitert.

```
#include ".././.././././constant/format/cybol/logic/custom_logic_cybol_format.c"
...

//
// custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_integer_equal((void*) &r, p1, (void*) PRETEXT_CUSTOM_LOGIC_CYBOI_FORMAT);

    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

        append_item_element(p0, (void*) PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT,
            (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
            (void*) PRETEXT_CUSTOM_LOGIC_CYBOL_FORMAT_COUNT,
            (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);
    }
}
```

Datei: Src/executor/representer/serialiser/cybol/format_cybol_serialiser.c

4.5 Funktion aufrufen

Im Operations-Händler muss nun angegeben werden, welche Funktion aufgerufen werden soll. Hier sei vorweg genommen, dass im folgenden noch die Funktion „appy_custom“ angelegt wird, welche die folgende Anpassung im Operation-Händler (Datei „operation_handler.c“) aufruft.

...

```
#include "../applicator/custom/custom.c"
...
//
//custom
//

if (r == *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    compare_integer_equal((void*) &r, p8, (void*) PRETEXT_CUSTOM_LOGIC_CYBOI_FORMAT);

    if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {
        apply_custom(p0, p1, p3);
    }
}
...
```

Datei: src/controller/handler/operation_handler.c

4.6 Die Funktion

Zunächst wird ein Applikator benötigt. Dies ist eine Funktion, welche vom Operations-Händler aufgerufen wird die Knoten der Parameter der Funktion ausliest und dann die eigentliche Funktion aufruft. Hierfür wird die Datei „custom.c“ angelegt.

```
#ifndef APPLY_CUSTOM_SOURCE
#define APPLY_CUSTOM_SOURCE

#include "../constant/model/cyboi/state/integer_state_cyboi_model.c"
#include "../constant/model/cyboi/state/pointer_state_cyboi_model.c"
#include "../constant/type/cyboi/state_cyboi_type.c"
#include "../executor/modifier/copier/array_copier.c"
#include "../executor/accessor/knowledge_getter/knowledge_part_getter.c"
#include "../executor/accessor/name_getter/array_name_getter.c"
#include "../executor/modifier/overwriter/part_overwriter.c"

#include "../constant/name/cybol/operation/custom_operation_cybol_name.c"
#include "../executor/custom/pretext.c"

void apply_custom(void* p0, void* p1, void* p2) {
    // The result part.
    void* r = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The pretext part.
    void* p = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The type part.
    void* t = *NULL_POINTER_STATE_CYBOI_MODEL;

    // Get result part.
    get_part_knowledge((void*) &r, p0, (void*) RESULT_CUSTOM_OPERATION_CYBOL_NAME,
        (void*) RESULT_CUSTOM_OPERATION_CYBOL_NAME_COUNT, p1, p2);
    // Get pretext part.
    get_part_knowledge((void*) &p, p0, (void*) PRETEXT_CUSTOM_OPERATION_CYBOL_NAME,
        (void*) PRETEXT_CUSTOM_OPERATION_CYBOL_NAME_COUNT, p1, p2);
    // Get type part.
    get_part_knowledge((void*) &t, p0, (void*) TYPE_CUSTOM_OPERATION_CYBOL_NAME,
        (void*) TYPE_CUSTOM_OPERATION_CYBOL_NAME_COUNT, p1, p2);

    // Calculate result by applying operation.
    custom_pretext(r, p, t);
}

/* APPLY_CUSTOM_SOURCE */
#endif
```

Datei: Src/applicator/custom/custom.c

Der Applicator ruft die Funktion „custom_pretext“ auf. Diese bekommt die Knoten der Parameter „result“, „pretext“ und „type“ übergeben. Die Funktion steht in der Datei „pretext.c“, welche im Folgenden angelegt wird.

```
#ifndef Exec_CUSTOM_ADD_SOURCE
#define Exec_CUSTOM_ADD_SOURCE
```

```

#include "../constant/model/cyboi/state/integer_state_cyboi_model.c"
#include "../constant/model/cyboi/state/pointer_state_cyboi_model.c"
#include "../constant/name/cyboi/state/part_state_cyboi_name.c"
#include "../constant/type/cyboi/state_cyboi_type.c"
#include "../executor/modifier/copier/array_copier.c"
#include "../executor/modifier/copier/integer_copier.c"

#include "../executor/modifier/emptier/item_emptier.c"
#include "../executor/modifier/inserter/item_inserter.c"

/**
 * Calculates all elements of the result- with those of the operand part.
 *
 * @param p0 the result part
 * @param p1 the pretext part
 * @param p2 the type part
 */
void custom_pretext(void* p0, void* p1, void* p2) {

    // The result model.
    void* rm = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The operand model.
    void* pm = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The type part model item.
    void* tm = *NULL_POINTER_STATE_CYBOI_MODEL;

    // The result data, count.
    void* rmd = *NULL_POINTER_STATE_CYBOI_MODEL;
    void* rmc = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The operand data, count.
    void* pmc = *NULL_POINTER_STATE_CYBOI_MODEL;
    void* pmd = *NULL_POINTER_STATE_CYBOI_MODEL;
    // The type part model item data.
    void* tmd = *NULL_POINTER_STATE_CYBOI_MODEL;

    // Get result model.
    copy_array_forward((void*) &rm, p0, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) MODEL_PART_STATE_CYBOI_NAME);
    // Get result model data, count.
    copy_array_forward((void*) &rmd, rm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) DATA_ITEM_STATE_CYBOI_NAME);
    copy_array_forward((void*) &rmc, rm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) COUNT_ITEM_STATE_CYBOI_NAME);

    // Get operand model.
    copy_array_forward((void*) &pm, p1, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) MODEL_PART_STATE_CYBOI_NAME);
    // Get operand model data, count.
    copy_array_forward((void*) &pmd, pm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) DATA_ITEM_STATE_CYBOI_NAME);
    copy_array_forward((void*) &pmc, pm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) COUNT_ITEM_STATE_CYBOI_NAME);

    // Get type part model.
    copy_array_forward((void*) &tm, p2, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) MODEL_PART_STATE_CYBOI_NAME);
    // Get type part model data.
    copy_array_forward((void*) &tmd, tm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) DATA_ITEM_STATE_CYBOI_NAME);

    //temporary copy of result item
    void* trm = *NULL_POINTER_STATE_CYBOI_MODEL;
    void* trmd = *NULL_POINTER_STATE_CYBOI_MODEL;
    void* trmc = *NULL_POINTER_STATE_CYBOI_MODEL;
    allocate_item((void*) &trm, (void*) NUMBER_1_INTEGER_STATE_CYBOI_MODEL, (void*)
INTEGER_NUMBER_STATE_CYBOI_TYPE);
    overwrite_item_element(trm, (void*) rmd, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE, (void*) rmc, (void*)
VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*)
TRUE_BOOLEAN_STATE_CYBOI_MODEL, (void*) DATA_ITEM_STATE_CYBOI_NAME);
    copy_array_forward((void*) &trmd, trm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) DATA_ITEM_STATE_CYBOI_NAME);
    copy_array_forward((void*) &trmc, trm, (void*) POINTER_STATE_CYBOI_TYPE, (void*) PRIMITIVE_STATE_CYBOI_MODEL_COUNT,
(void*) VALUE_PRIMITIVE_STATE_CYBOI_NAME, (void*) COUNT_ITEM_STATE_CYBOI_NAME);

    //result Item leeren
    empty_item(rm, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE);

```

```

//pretext einfügen
append_item_element(rm, (void*)pmd, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE, pmc, (void*)
NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

int r = *FALSE_BOOLEAN_STATE_CYBOI_MODEL;

//wenn Type = custom/vat
compare_integer_equal((void*)&r, tmd, (void*) VAT_CUSTOM_STATE_CYBOI_TYPE);

if (r != *FALSE_BOOLEAN_STATE_CYBOI_MODEL) {

    //"Mehrwertsteuer" zum result Item hinzufügen
    wchar_t MWST_ARRAY[] = {L' ',L'M', L'e', L'h',L'r',L'w', L'e', L'r', L't',L's', L't', L'e', L'u',L'e', L'r'};
    wchar_t* MWST = MWST_ARRAY;
    static int* MWST_COUNT = NUMBER_15_INTEGER_STATE_CYBOI_MODEL_ARRAY;
    append_item_element(rm, (void*)MWST, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
        MWST_COUNT, (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

}

//Leerzeichen am result Item anfügen
wchar_t Leerzeichen_ARRAY[] = {L' '};
wchar_t* Leerzeichen = Leerzeichen_ARRAY;
append_item_element(rm, (void*)Leerzeichen, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE,
    (void*) NUMBER_1_INTEGER_STATE_CYBOI_MODEL, (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

//Alten Wert des result items an aktuelles anfügen
append_item_element(rm, (void*)trmd, (void*) WIDE_CHARACTER_TEXT_STATE_CYBOI_TYPE, trmc,
    (void*) NUMBER_0_INTEGER_STATE_CYBOI_MODEL);

}

/* Exec_CUSTOM_ADD_SOURCE */
#endif

```

Datei: Src/executor/custom/pretext.c

Diese umfangreiche Funktion erledigt folgendes:

- zu den übergebenen Knoten „result“, „pretext“ und „type“ den Inhalt herausuchen
- von „result“ eine Kopie anlegen und anschließen „result“ leeren
- den „pretext“ in „result“ einfügen (im Beispiel „inkl“)
- wenn der Type gleich „custom/vat“, dann die Zeichenfolge „Mehrwertsteuer“ in „result“ anhängen
- auf der Kopie von „result“ den ursprünglichen Wert holen (im Beispiel „120,50“) und an „result“ anhängen

Damit ist die Funktion nun fertig.

Hier ein paar Beispiel-Einstellungen in der cybol-Beispieldatei und die darugehörige Ausgabe des Programms:

```

...
<part name="do_Pretext" channel="inline" encoding="utf-8" language="text/cybol" format="custom/pretext" model="">
  <property name="result" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model="".betrag"/>
  <property name="pretext" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="inkl"/>
  <property name="type" channel="inline" encoding="utf-8" language="text/cybol" format="meta/type" model="custom/vat"/>
</part>
...

```

```

inkl Mehrwertsteuer 120,50 EUR
Information: Exit cyboi normally.

```

```
...
<part name="do_Pretext" channel="inline" encoding="utf-8" language="text/cybol" format="custom/pretext" model="">
  <property name="result" channel="inline" encoding="utf-8" language="text/cybol" format="path/knowledge" model=".betrag"/>
  <property name="pretext" channel="inline" encoding="utf-8" language="text/cybol" format="text/plain" model="Preis:"/>
  <property name="type" channel="inline" encoding="utf-8" language="text/cybol" format="meta/type" model="text/plain"/>
</part>
...
```

Preis: 120,50 EUR

Information: Exit cyboi normally.

ENDE